

Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology



Department of Computer Science and Engineering

Course Code : CSE 4106

Course Title : Digital Image Processing Sessional

Lab Report

Experiment Name: Fundamental Techniques in Digital Image Enhancement and Transformation

Submission Date: July 28, 2025

Submitted By:

Eazdan Mostafa Rafin

Roll: 2003055

Section : A

Submitted To:

S. M. Mahedy Hasan

Assistant Professor

Dept. of CSE, RUET

Fundamental Techniques in Digital Image Enhancement and Transformation

Objective

To explore and analyze essential image processing techniques for enhancing visual quality and extracting valuable information from digital images.

1 Introduction

Digital image processing plays a crucial role in modern technology, impacting diverse fields from medical imaging and satellite photography to everyday smartphone apps. At its core, digital image processing involves manipulating visual data to improve clarity, extract information, or prepare images for further analysis. This report explores key fundamental techniques that significantly enhance or transform images, providing both visual appeal and functional clarity.

Firstly, we delve into image negatives, a technique that inverts pixel intensities, creating striking visual contrasts and often highlighting subtle details invisible in the original image. Subsequently, we explore log transformation, a method particularly effective in managing images with wide-ranging pixel values by compressing dynamic range and revealing hidden details in dark regions. The report further examines gamma correction, an essential approach for adjusting brightness and contrast, ensuring images appear natural and perceptually accurate on various display devices.

Additionally, the concept of min-max normalization is introduced, emphasizing its role in rescaling pixel intensities to a defined range, thus improving uniformity and consistency across images. We also analyze thresholding, a powerful segmentation tool that partitions images into distinct regions, simplifying the identification and extraction of meaningful features. Lastly, the report highlights intensity slicing, a selective enhancement technique useful for emphasizing specific intensity ranges, thereby facilitating focused analysis.

2 Methodology

All processing steps are implemented in Python using OpenCV and NumPy. The input image is first loaded in grayscale using OpenCV, and Matplotlib is used for visualization. Then, each of the following subsections applies a specific intensity transformation to the image and displays the result.

2.1 Image Negative

```
rows, cols = img.shape
negative_img = np.zeros_like(img)
for i in range(rows):
    for j in range(cols):
        negative_img[i, j] = 255 - img[i, j]

plt.figure(figsize=(12, 8))

plt.subplot(1, 2, 1)
```

```
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(negative_img, cmap='gray')
plt.title('Negative')
plt.axis('off')
```



Figure 1: Original image and its negative.

The negative transformation inverts the intensity of each pixel, producing a photographic negative of the original image. High-valued (bright) pixels become low-valued (dark) in the negative image, and vice versa, thereby reversing the brightness and contrast of the original.

2.2 Log Transformation

```
max_value = np.max(img)
min_value = np.min(img)
print(f"Max pixel value: {max_value}, Min pixel value: {min_value}")
c = 255 / np.log(1 + np.max(img))
log_transformed = c * np.log(1 + img)
log_transformed = np.array(log_transformed, dtype=np.uint8)

plt.figure(figsize=(12, 8))

plt.subplot(1, 2, 1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(log_transformed, cmap='gray')
plt.title('Log-Transformed Image')
plt.axis('off')
```

Max pixel value: 248, Min pixel value: 22



Figure 2: Original image and its log-transformed version.

Applying a logarithmic transformation expands the range of dark pixel values while compressing the brighter pixel values. This results in an image where details in darker regions are enhanced, as the log curve brightens low-intensity values more significantly than high-intensity ones.

2.3 Gamma Correction

```
gamma = 2
gamma_corrected = np.array(255 * (img / 255) ** gamma, dtype=np.uint8)

plt.figure(figsize=(12, 8))

plt.subplot(1, 2, 1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(gamma_corrected, cmap='gray')
plt.title('Gamma-Corrected Image')
plt.axis('off')
```

Gamma correction (with $\gamma = 2$ here) adjusts the overall brightness and contrast of the image using a power-law function. Because $\gamma > 1$, the mid-tone and dark regions of the image are made relatively darker (compared to the original), illustrating how different gamma values can be used to brighten or darken an image.

2.4 Min-Max Normalization

```
r_min = np.min(img)
r_max = np.max(img)
print(f"Range of pixel values: {r_min} to {r_max}")
min_max_scaled = ((img - r_min) / (r_max - r_min)) * 255
min_max_scaled = np.array(min_max_scaled, dtype=np.uint8)

plt.figure(figsize=(12, 8))
```



Figure 3: Original image and its gamma corrected version.

```
plt.subplot(1, 2, 1)
plt.imshow(img, cmap='gray')
plt.title('Original-Image')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(min_max_scaled, cmap='gray')
plt.title('Min-Max-Normalized-Image')
plt.axis('off')
```

Range of pixel values: 22 to 248



Figure 4: Original image and its min-max normalized version.

Min-max intensity normalization scales the pixel values linearly to span the full 0 to 255 range. In the normalized image, the darkest pixel becomes black (0) and the brightest pixel becomes white (255), increasing the overall contrast compared to the original image.

2.5 Thresholding

```
T = 127
threshold_img = np.where(img > T, 255, 0).astype(np.uint8)

plt.figure(figsize=(12, 8))

plt.subplot(1, 2, 1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(threshold_img, cmap='gray')
plt.title('Thresholded Image')
plt.axis('off')
```



Figure 5: Original image and the thresholded binary image.

Thresholding converts the grayscale image into a binary image. All pixels with intensity above the threshold (127 in this case) are set to white (255), while those below are set to black (0), effectively segmenting the image into foreground and background regions.

2.6 Intensity Slicing

```
# Define slicing range
lower = 100
upper = 150

# 1. Intensity Slicing Without Background Suppression
sliced_without_bg = img.copy()
sliced_without_bg[(img >= lower) & (img <= upper)] = 255

# 2. Intensity Slicing With Background Suppression
sliced_with_bg = np.zeros_like(img)
sliced_with_bg[(img >= lower) & (img <= upper)] = 255
```

```

# Display all images using subplots
plt.figure(figsize=(10, 6))

plt.subplot(1, 3, 1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(sliced_without_bg, cmap='gray')
plt.title('Sliced (No BG Suppression)')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(sliced_with_bg, cmap='gray')
plt.title('Sliced (With BG Suppression)')
plt.axis('off')

plt.tight_layout()
plt.show()

```



Figure 6: Original image (left), intensity-sliced image without background suppression (center), and with background suppression (right).

Intensity slicing highlights a specific range of intensities (100 to 150 here) in the image. In one output, pixels within this range are turned white in the context of the original image (no background suppression), and in another output, only those pixels are shown on a black background (background suppression), thereby isolating features that fall within the chosen intensity range.

3 Conclusion

Digital image processing techniques such as image negatives, log transformations, gamma correction, min-max normalization, thresholding, and intensity slicing collectively offer powerful tools for enhancing image clarity, visibility, and interpretability. Each method uniquely contributes to improving image quality, extracting meaningful information, and addressing specific image-related challenges. Understanding and applying these techniques can significantly elevate the effectiveness of visual analysis across diverse fields, including medical imaging, remote sensing, and multimedia technologies. As digital imaging continues to evolve, proficiency in these fundamental methods remains essential for harnessing the full potential of visual data in both scientific research and everyday applications.